

ACM-ICPC 个人算法模板

算法竞赛常用代码与知识点汇总

整理人：halfpoint

适用场景：ACM-ICPC、CCPC、蓝桥杯、牛客算法竞赛等

代码语言：C++

目录

章节	标题	页码
一	字符串哈希 (hash_sum)	3

章节	标题	页码
二	树的基础数据结构与算法	4
2.1	树的基础概念	4
2.2	前中后序遍历	5
2.3	并查集	8
2.4	MST（最小生成树）	9
2.4.1	Prim算法	9
2.4.2	Kruskal算法	11
2.5	LCA（最近公共祖先）	13
2.5.1	树链剖分	13
2.5.2	倍增	14
三	单调队列 & 单调栈 核心笔记	15
3.1	单调队列	15
3.2	单调栈	16
四	ST表（倍增）	17
五	字符串算法（KMP + Manacher）	19
5.1	KMP算法	19
5.2	Manacher（马拉车算法）	21
六	数论基础	22
6.1	费马小定理笔记	22
6.2	等比数列求和公式笔记	24

一、字符串哈希（hash_sum）

用法

维护字符串区间哈希值，快速比较子串是否相等、判断回文等。

- 预处理时间: $\mathcal{O}(n)$
- 查询时间: $\mathcal{O}(1)$

实现步骤

1. 生成随机基数 `base` (用 `mt19937_64` 生成, 防卡哈希)
2. 预处理 `base` 的幂次数组 `power`, 其中 `power[i] = base^i`
3. 预处理前缀哈希数组 `hash_pre`, 其中 `hash_pre[i]` 表示前 `i` 个字符的哈希值
4. 查询区间 `[l, r]` (1-based) 的哈希值: `hash_pre[r] - hash_pre[l-1] * power[r-l+1]`

完整模板

```
#include <bits/stdc++.h>
#include <random>
#include <chrono>
using namespace std;
using i64 = long long;
using u64 = unsigned long long;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

const int MAXN = 1e5 + 5;
u64 base;
u64 power[MAXN];
u64 hash_pre[MAXN];

void init_hash() {
    base = rng();
    power[0] = 1;
    for (int i = 1; i < MAXN; i++) {
        power[i] = power[i-1] * base;
    }
}

void build_hash(const string &s) {
    int n = s.size();
    for (int i = 0; i < n; i++) {
        hash_pre[i+1] = hash_pre[i] * base + s[i];
    }
}

u64 get_hash(int l, int r) {
    return hash_pre[r] - hash_pre[l-1] * power[r - l + 1];
}

int main() {
    init_hash();
    string s;
    cin >> s;
    build_hash(s);
    int l = 2, r = 4;
    cout << get_hash(l, r) << endl;
    return 0;
}
```


二、树的基础数据结构与算法

2.1 树的基础概念

树的定义

树的定义

n 个结点构成的有限集合

n=0 时：称为空树

n>0 时：

有且只有一个根结点

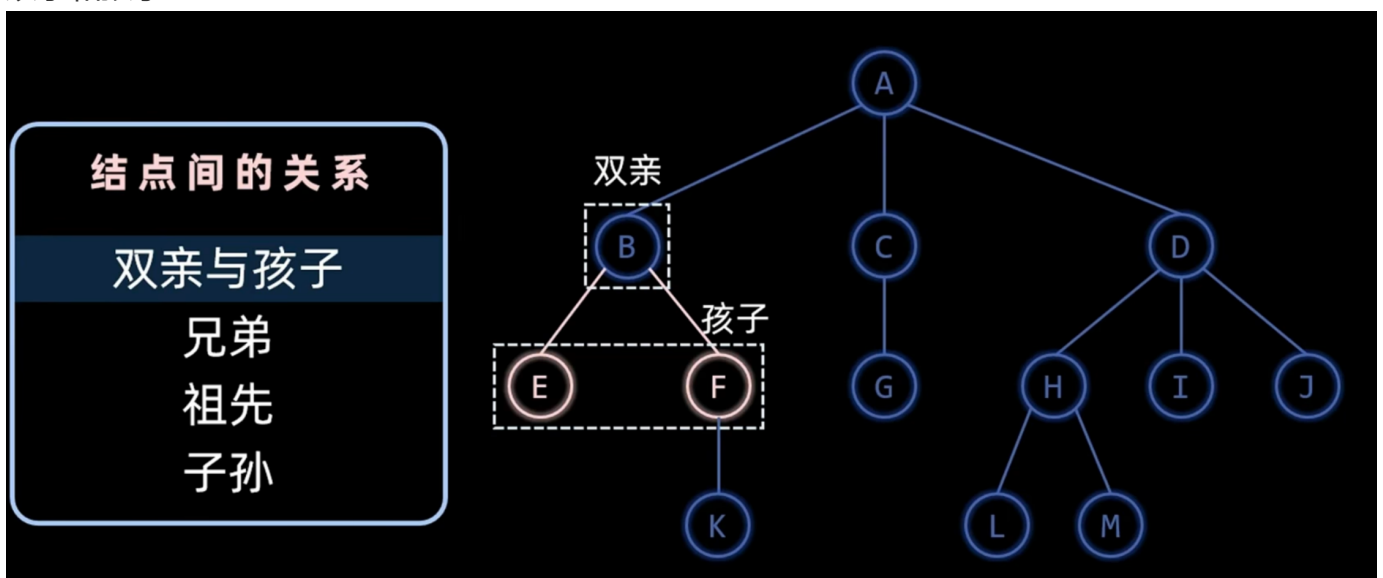
其余结点可分为 m 个互不相交的子集，每个子集也是树，被称为根结点的子树

根结点

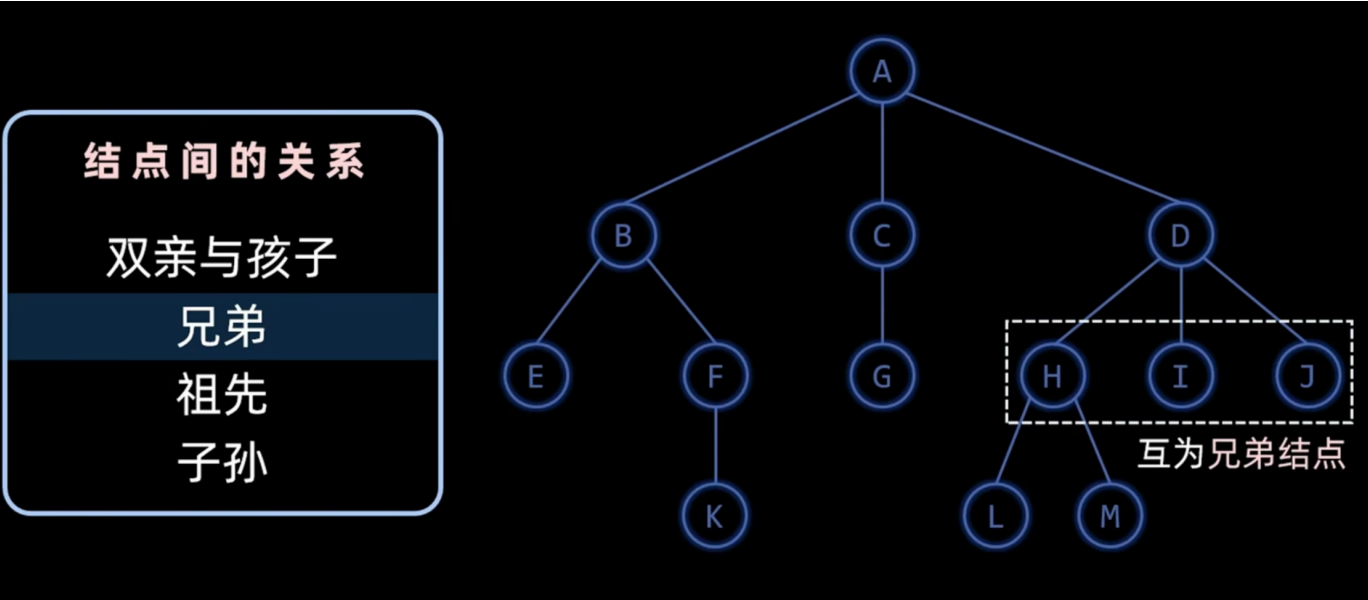
m = 3

结点间的关系

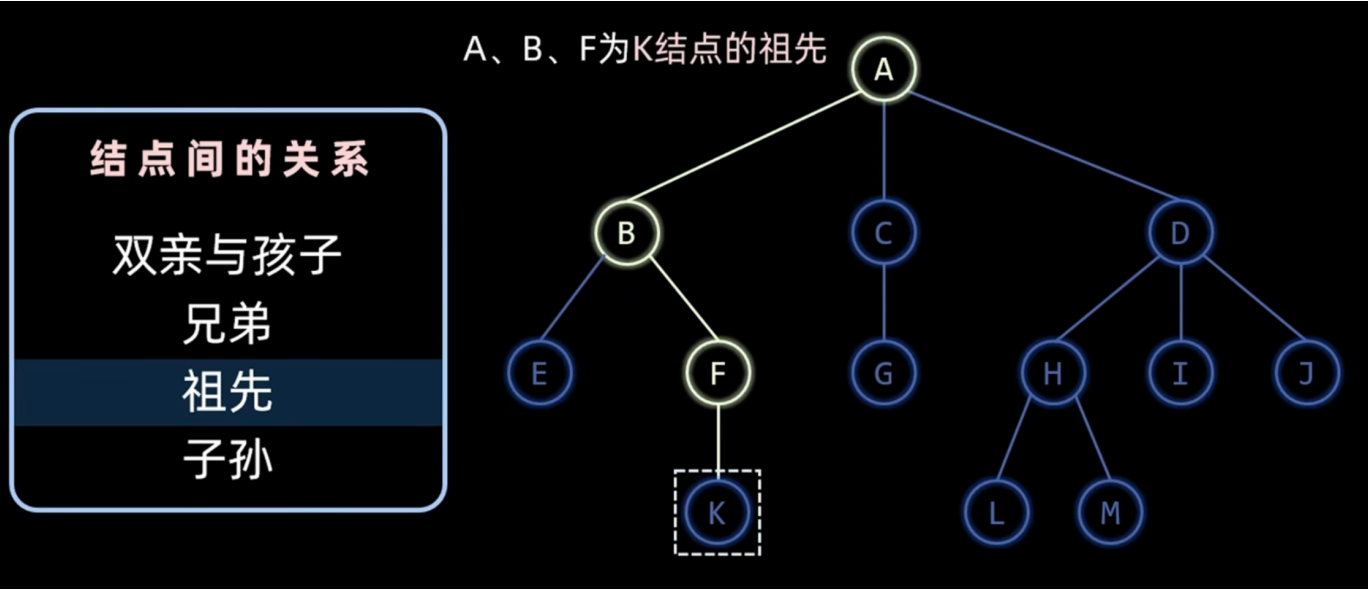
- 双亲和孩子



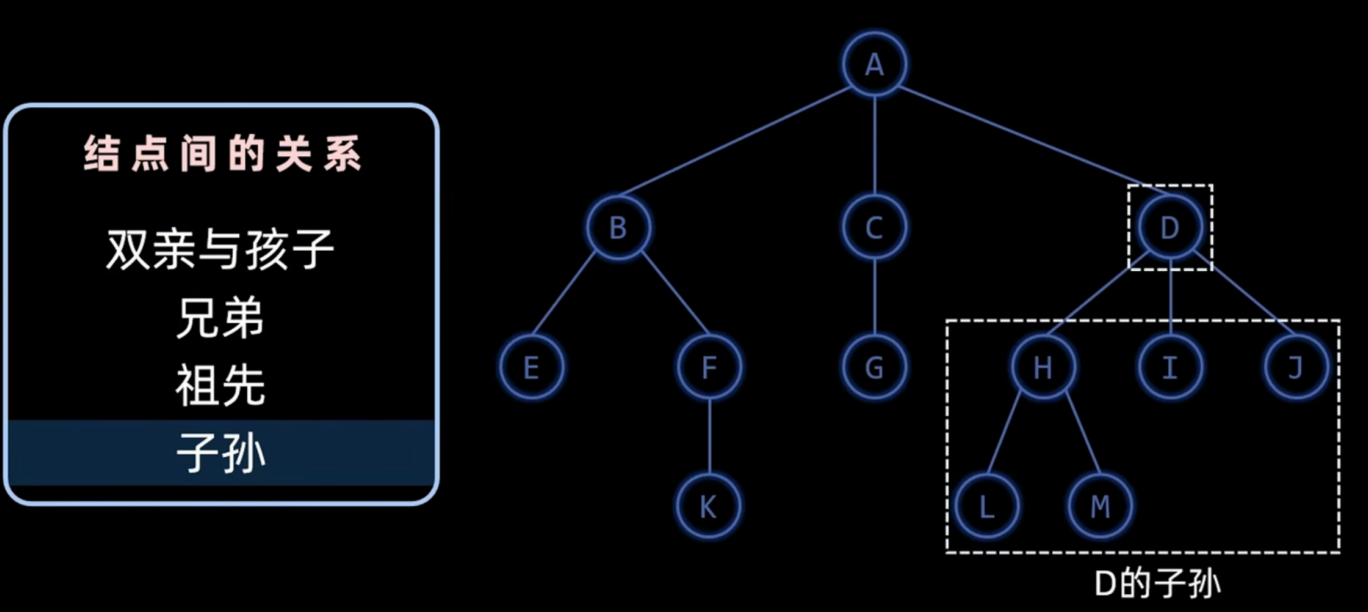
- 兄弟



• 祖先

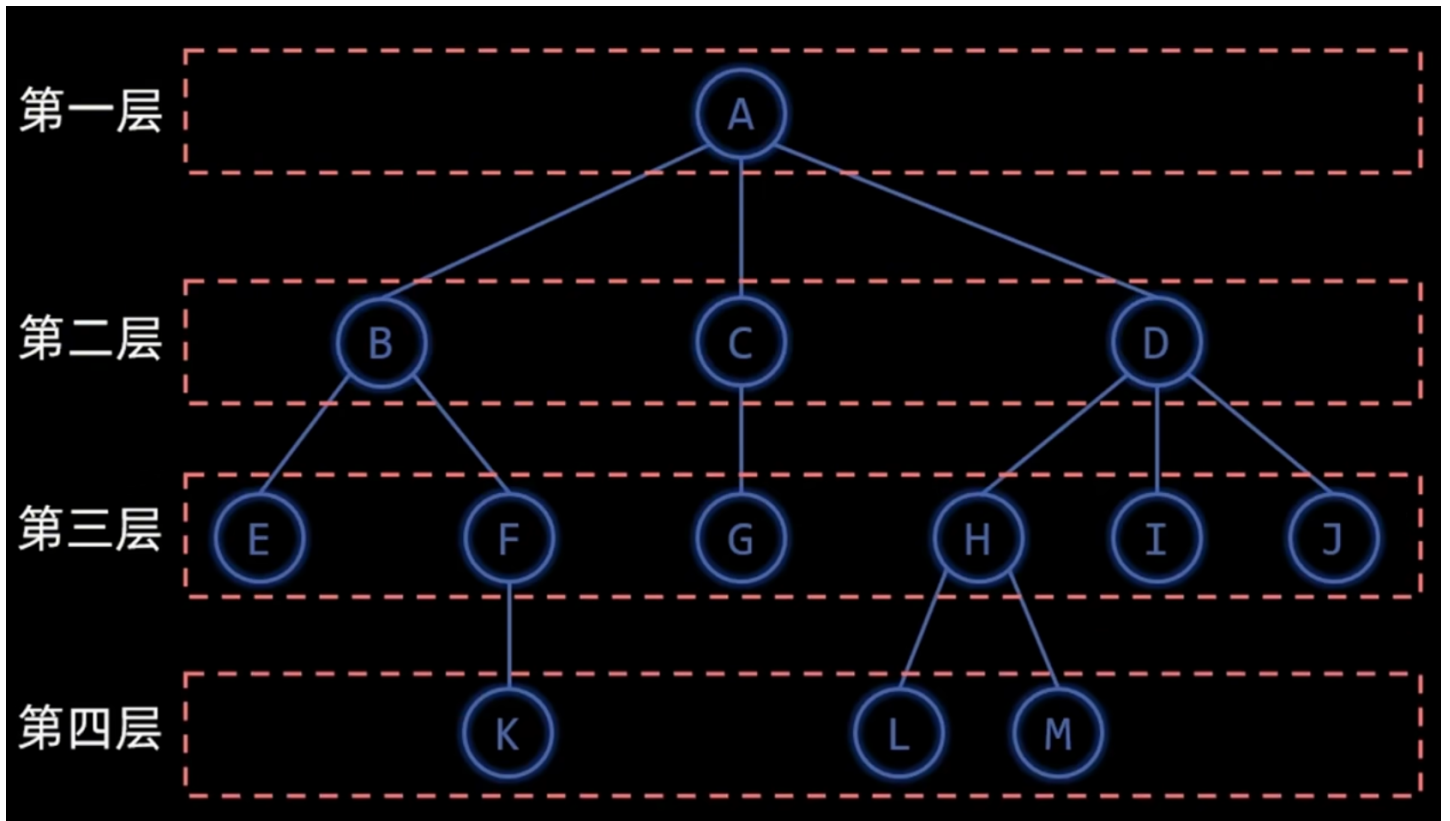


• 子孙



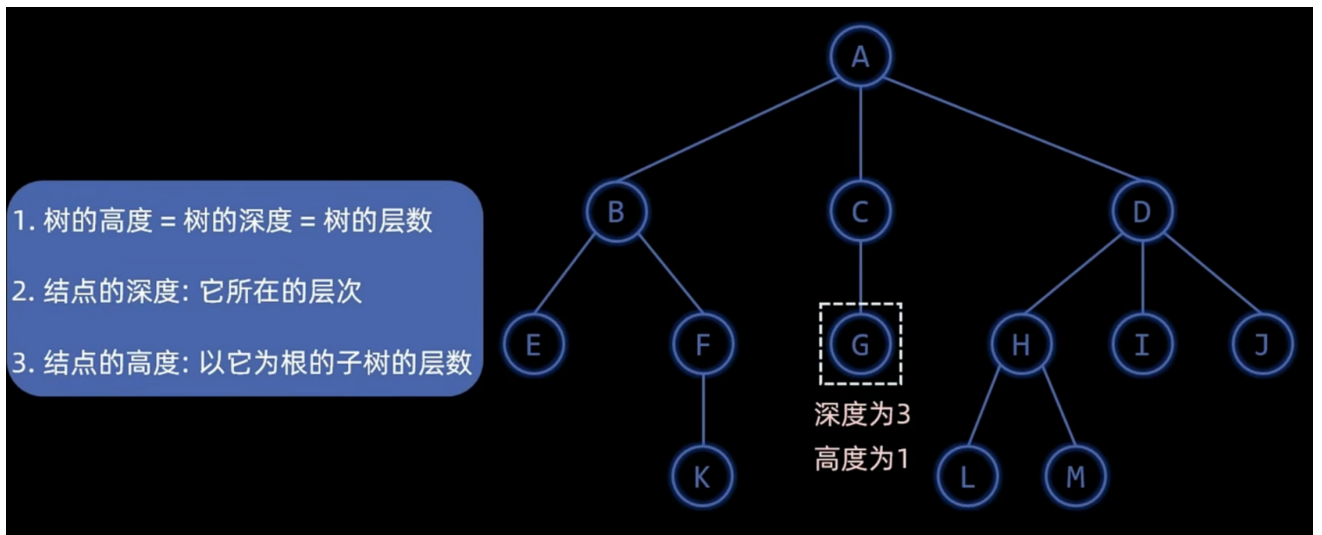
树的层数

下图共四层



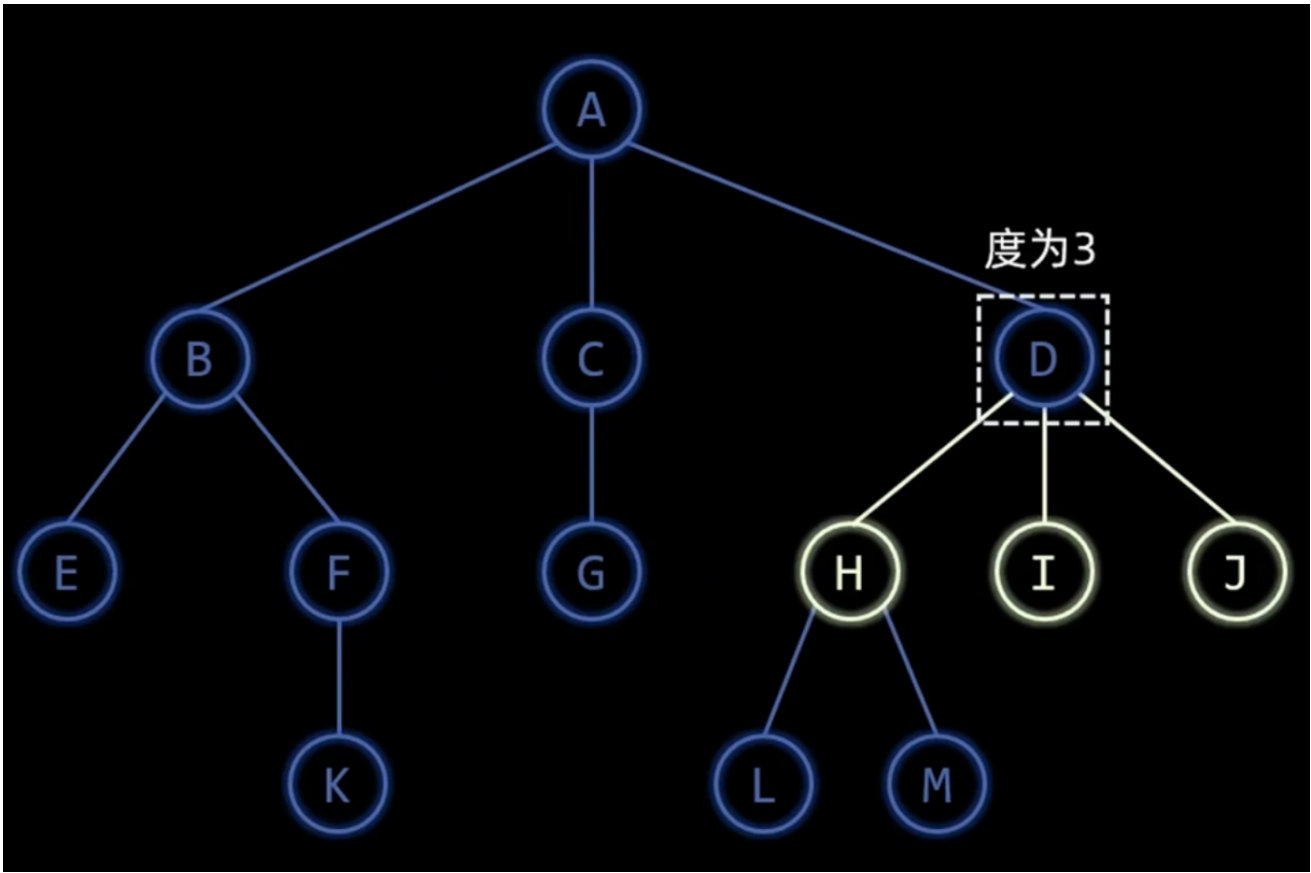
高度与深度

- 对树来说：高度 = 深度 = 层数
- 对结点来说：
 - 高度 = 从下往上
 - 深度 = 从上往下

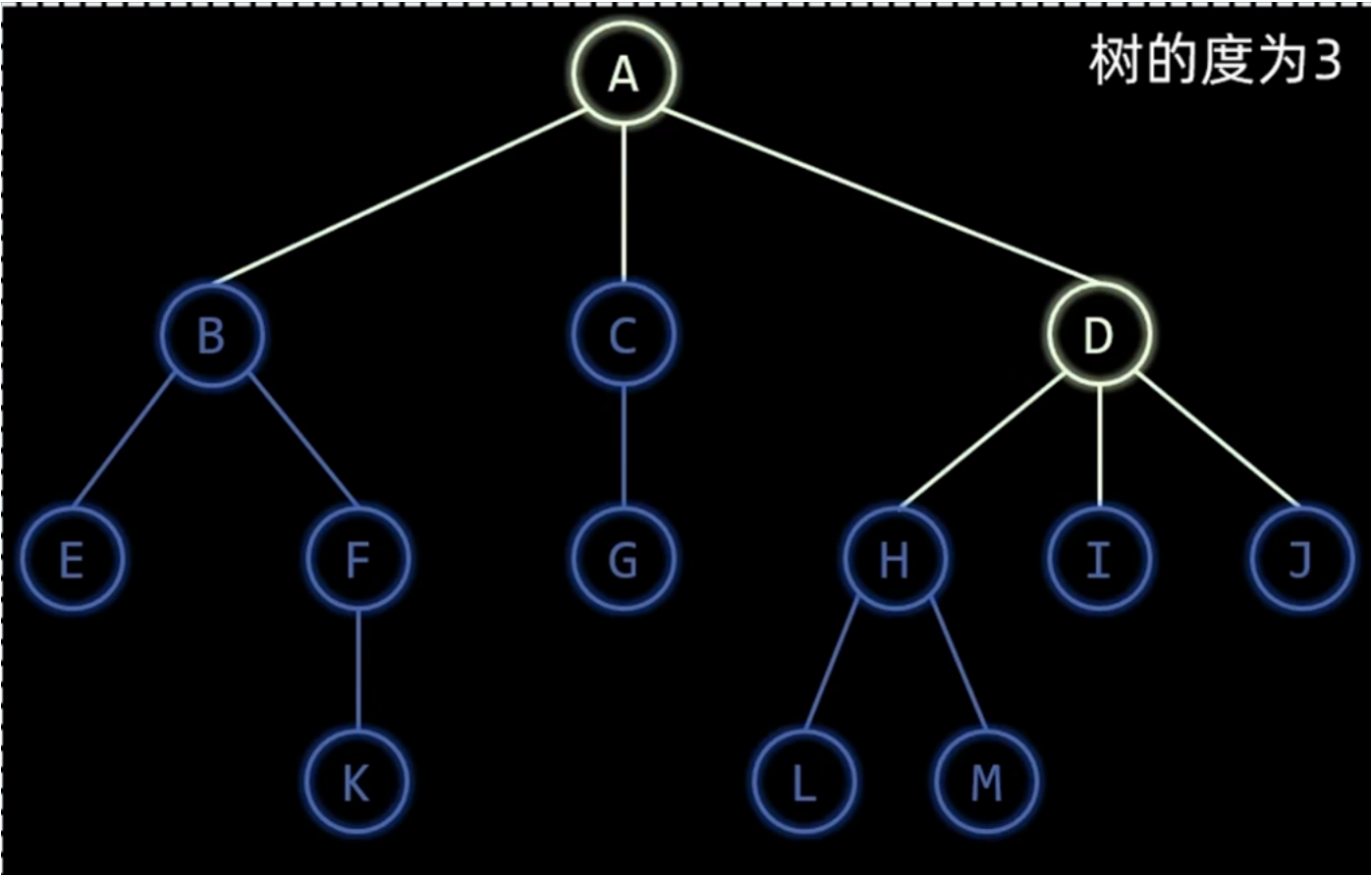


度

- 结点的度 = 该结点的分支数
 - 度 = 0 时，结点为叶结点
 - 度 > 0 时，称为分支结点



- 树的度 = 树里最大的结点度



m叉树和m度树的区别

叉是最大的上限，度是最大的下限

度为m的树和m叉树

度为m的树

m叉树

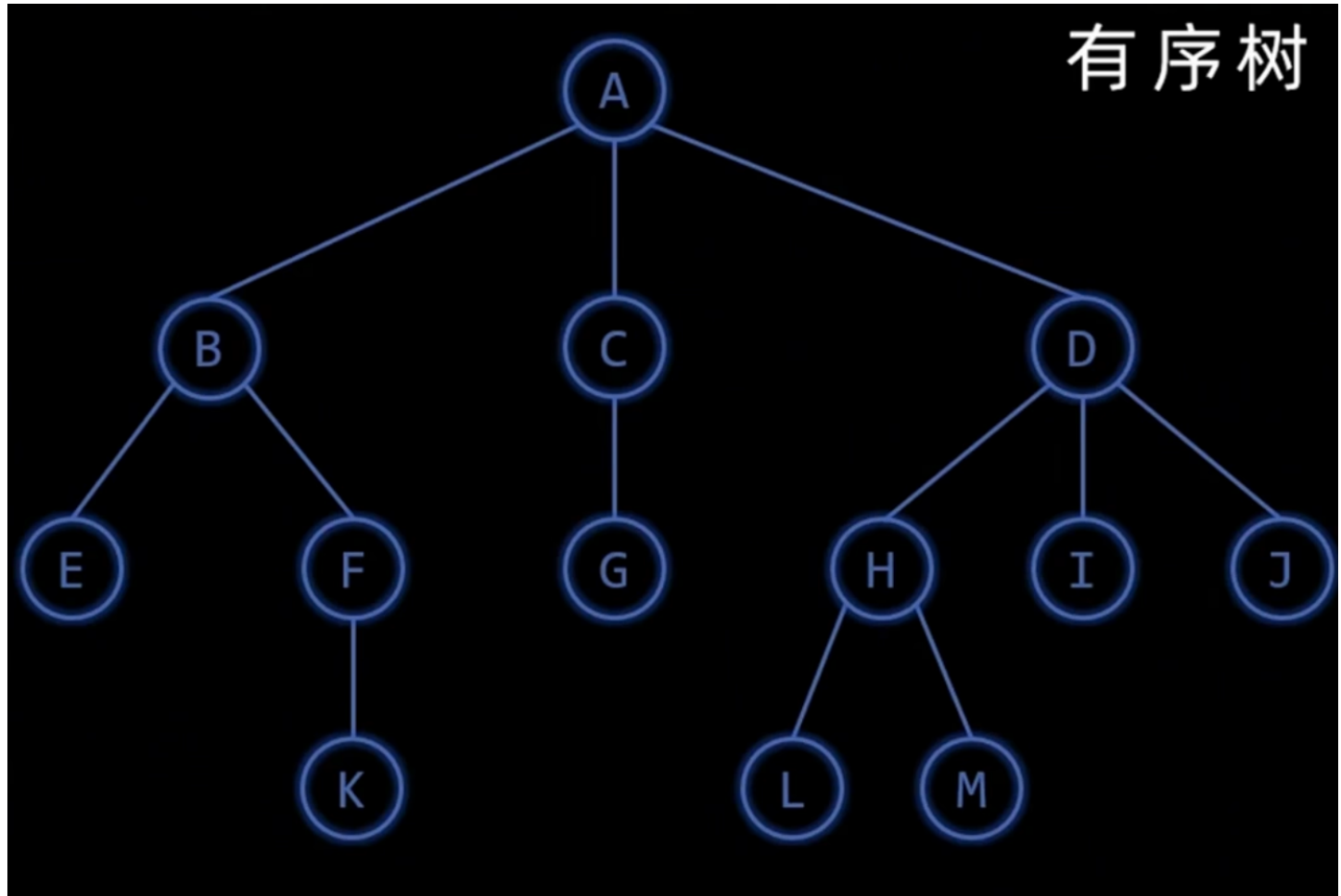
相同点：树里面结点的度最多为m

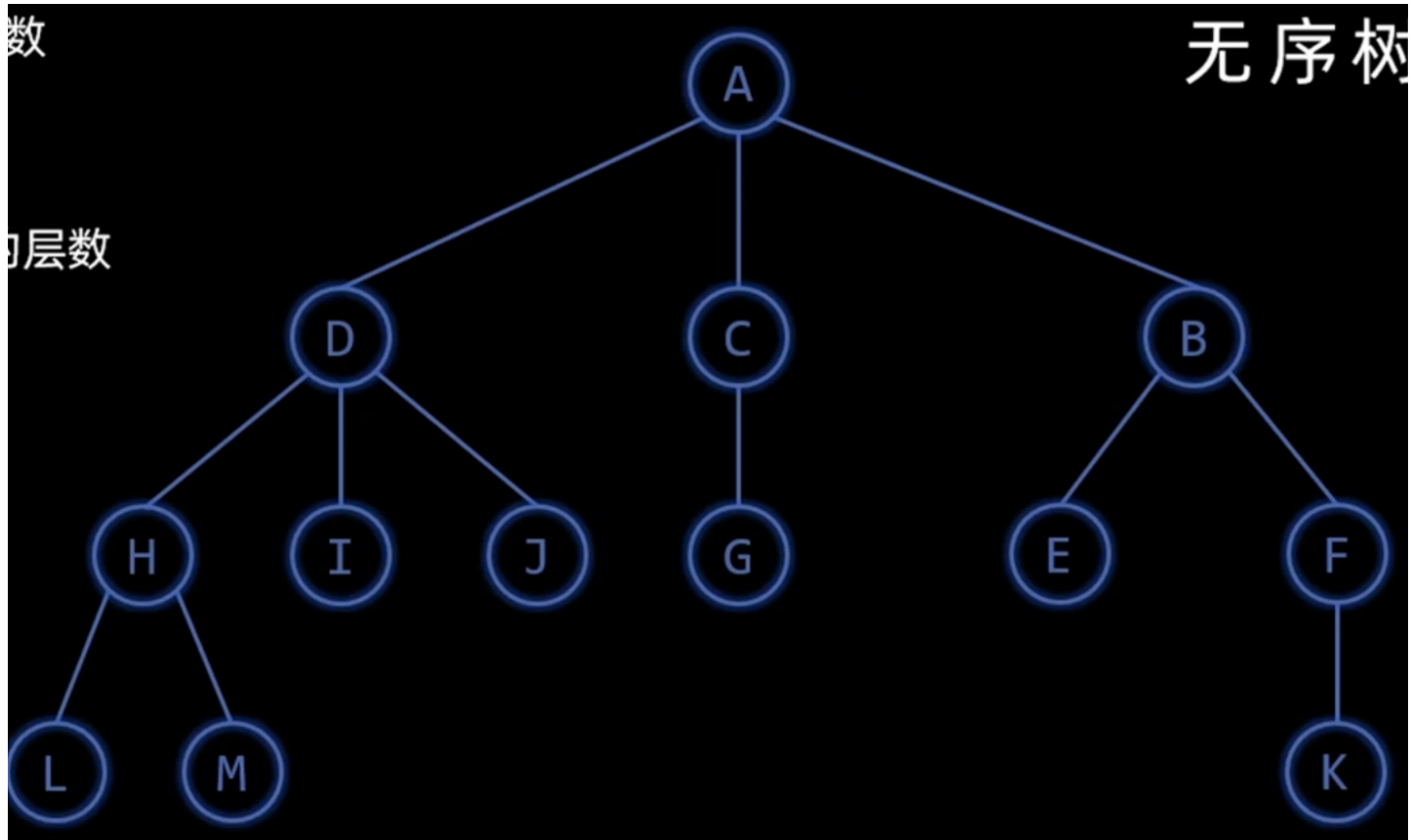
不同点：
度为m的树至少要有有一个结点的度等于m
m叉树所有结点的度可以都小于m，甚至可以是空树

```
graph TD; A((A)) --- B((B)); A --- C((C)); A --- D((D)); B --- E((E)); B --- F((F)); F --- K((K)); C --- G((G)); D --- H((H)); D --- I((I)); D --- J((J)); H --- L((L)); H --- M((M));
```

有序树和无序树

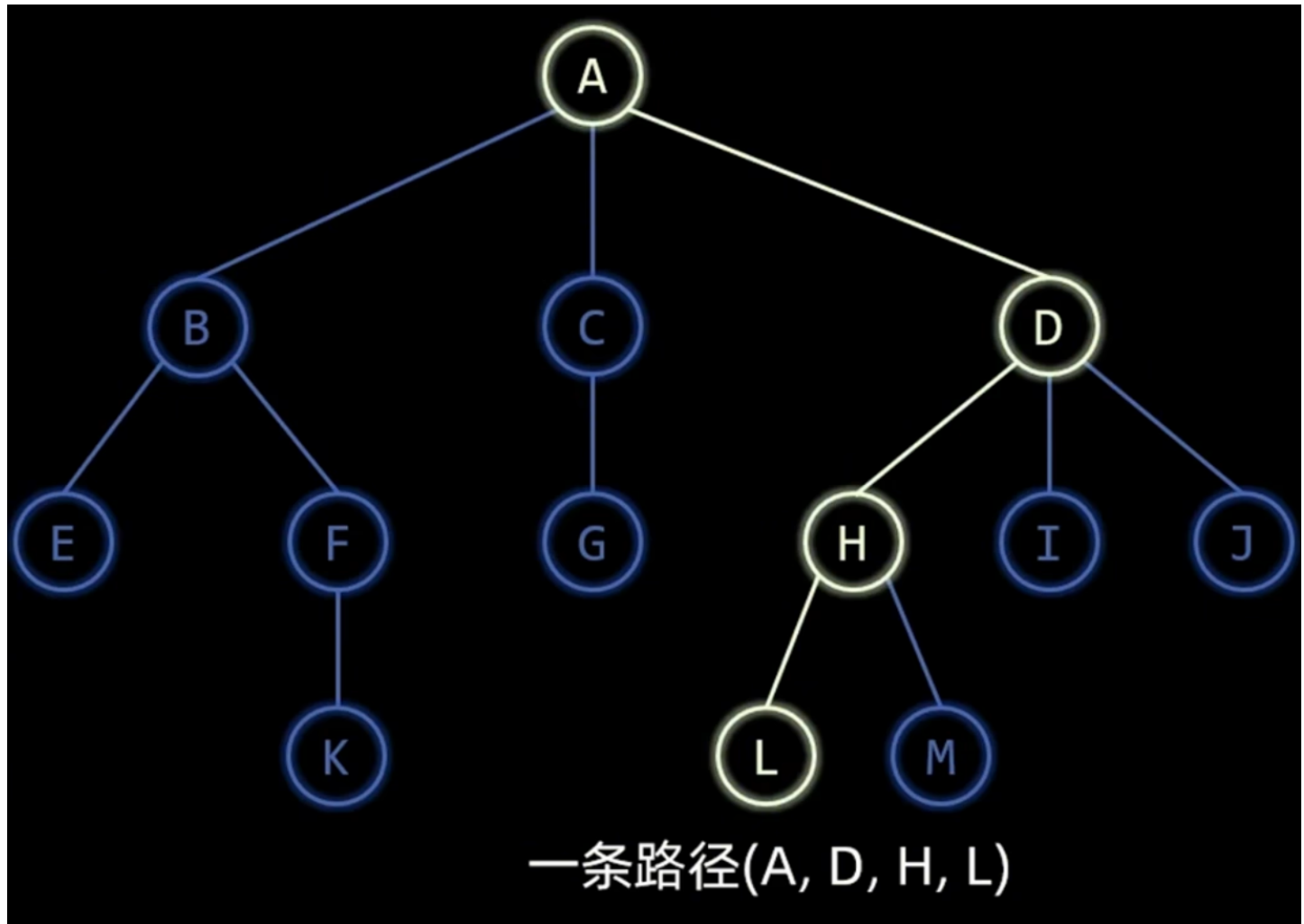
各结点子树从左到右有序





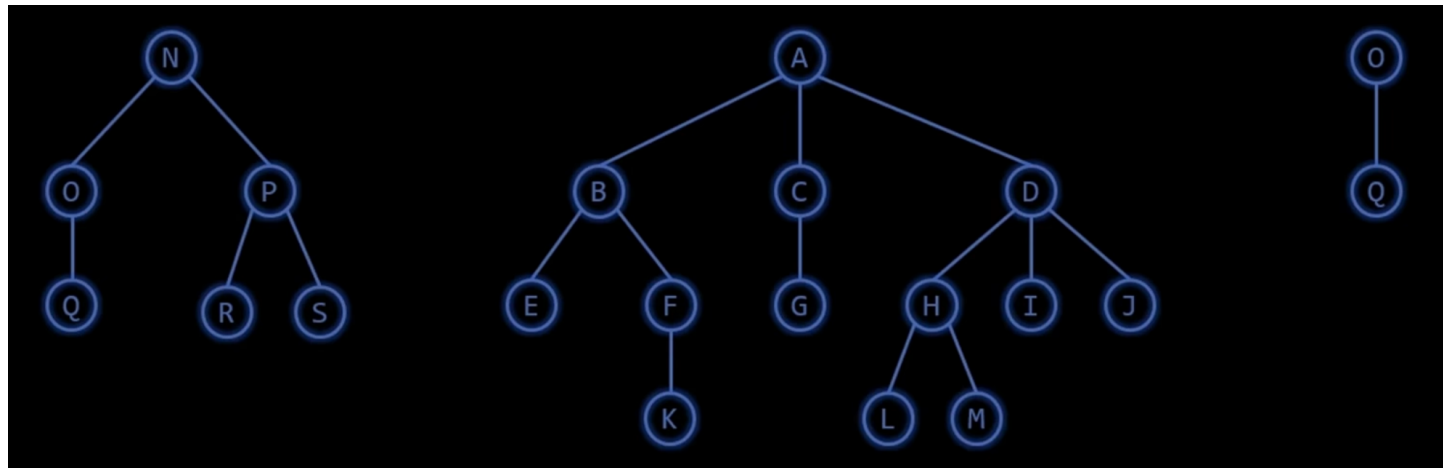
路径

从上到下经过的结点数



森林

$m(m \geq 0)$ 颗树的集合

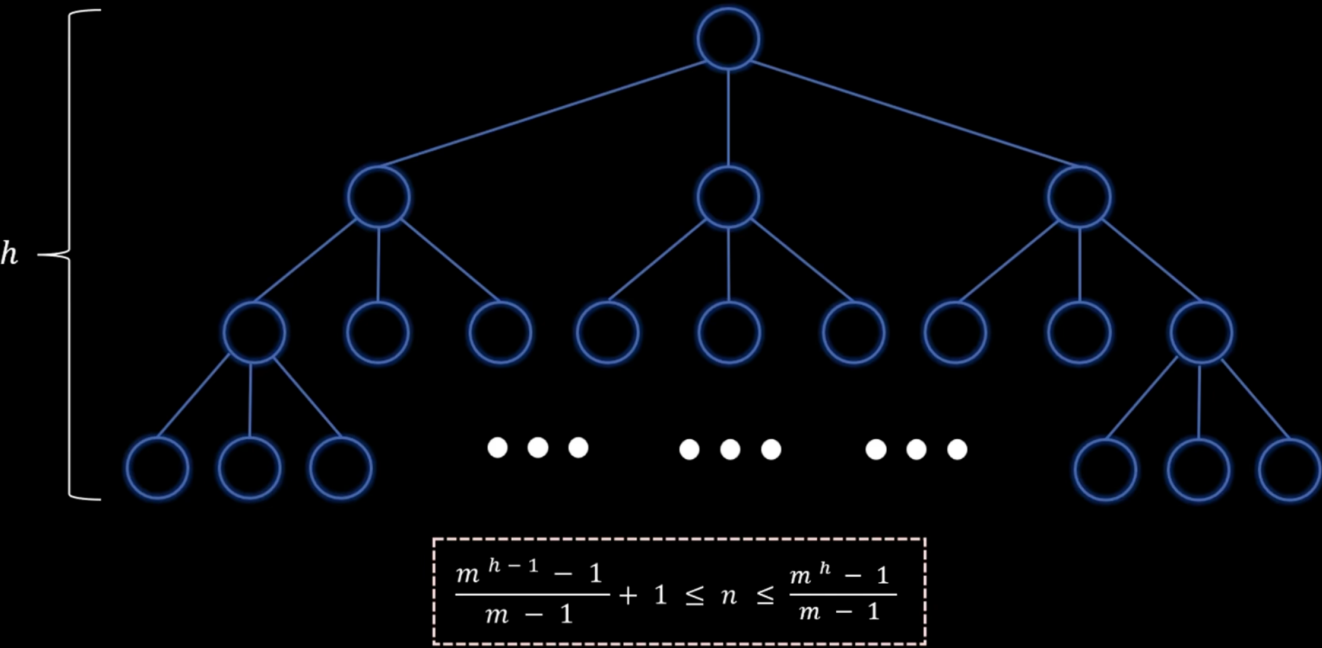


常见性质

- 1. 结点数 n = 边数 + 1 = 所有结点的度数和 + 1
- 2. m 叉树(设层数为 i)的每一层最多的结点数为 m^{i-1}
- 3. 具有 n 个结点的 m 叉树的最小高度

5. 具有 n 个结点的 m 叉树的最小高度为？

蓝不过海呀 bilibili



等价于： $\log_m(n(m-1)+1) \leq h < \log_m(n(m-1)+1) + 1$

2.2 前中后序遍历

用法

按照不同顺序访问树的所有结点，分为前序（根-左-右）、中序（左-根-右）、后序（左-右-根），可通过递归或栈实现。

实现步骤

- 前序：中左右，递归序 = 1
- 中序：左中右，递归序 = 2
- 后序：左右中，递归序 = 3

递归写法：

```
void 递归() {  
    if(head == NULL) {  
        break;  
    }  
    //输出head 前序  
    递归(head.left)  
    //输出head 中序  
    递归(head.right)  
    //输出head 后序  
  
}
```

完整模板（栈写法）


```
#include <iostream>
#include <vector>
#include <stack>
#include <algorithm>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

vector<int> preorderTraversal(TreeNode* root) {
    vector<int> result;
    if (!root) return result;
    stack<TreeNode*> stk;
    stk.push(root);
    while (!stk.empty()) {
        TreeNode* node = stk.top();
        stk.pop();
        result.push_back(node->val);
        if (node->right) stk.push(node->right);
        if (node->left) stk.push(node->left);
    }
    return result;
}

vector<int> inorderTraversal(TreeNode* root) {
    vector<int> result;
    stack<TreeNode*> stk;
    TreeNode* cur = root;
    while (cur || !stk.empty()) {
        while (cur) {
            stk.push(cur);
            cur = cur->left;
        }
        cur = stk.top();
        stk.pop();
        result.push_back(cur->val);
        cur = cur->right;
    }
    return result;
}

vector<int> postorderTraversal(TreeNode* root) {
```

```

vector<int> result;
if (!root) return result;
stack<TreeNode*> stk;
TreeNode* cur = root;
TreeNode* prev = nullptr;
while (cur || !stk.empty()) {
    while (cur) {
        stk.push(cur);
        cur = cur->left;
    }
    cur = stk.top();
    if (!cur->right || cur->right == prev) {
        result.push_back(cur->val);
        stk.pop();
        prev = cur;
        cur = nullptr;
    } else {
        cur = cur->right;
    }
}
return result;
}

```

```

vector<int> postorderTraversal_reverse(TreeNode* root) {
    vector<int> result;
    if (!root) return result;
    stack<TreeNode*> stk;
    stk.push(root);
    while (!stk.empty()) {
        TreeNode* node = stk.top();
        stk.pop();
        result.push_back(node->val);
        if (node->left) stk.push(node->left);
        if (node->right) stk.push(node->right);
    }
    reverse(result.begin(), result.end());
    return result;
}

```

```

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    cout << "前序遍历: ";
    for (int x : preorderTraversal(root)) cout << x << " ";
}

```

```
cout << "\n中序遍历: ";
for (int x : inorderTraversal(root)) cout << x << " ";
cout << "\n后序遍历: ";
for (int x : postorderTraversal(root)) cout << x << " ";
return 0;
}
```

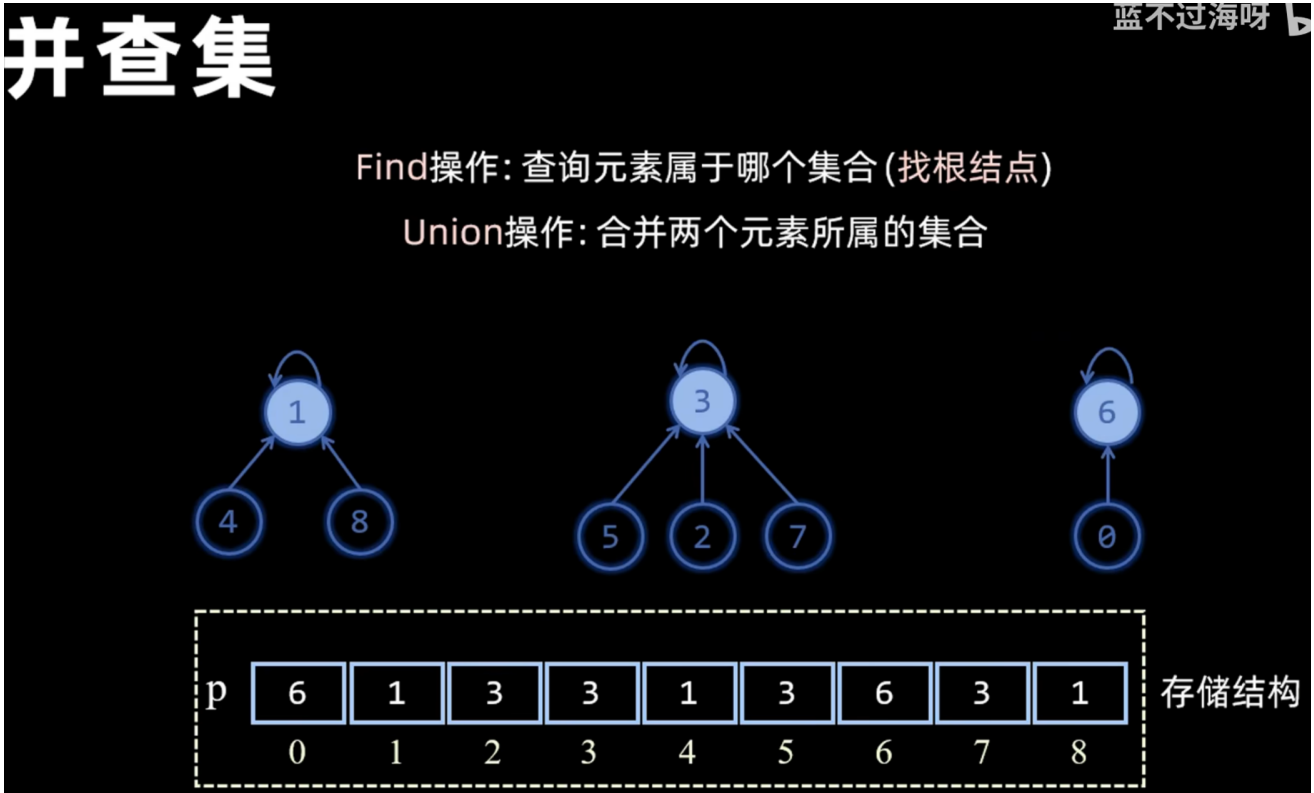
2.3 并查集

用法

用于动态维护和查询集合的合并与连通性，支持Find（查找根结点）和Union（合并集合）操作，路径压缩可优化查询效率至近似 $O(1)$ 。

实现步骤

1. 存储结构:
- 使用数组，下标表示结点，数组值表示该结点的父结点，根结点的父结点是自己



2. Find操作:
- 普通写法: 递归向上找根结点，不破坏原树结构
 - 路径压缩: 递归时让路径上的每个结点直接指向根，优化后续查询
3. Union操作: 找到两个元素的根结点，若不同则让一个根指向另一个根

完整模板

```
vector<int> P;  
  
int Find(int x) {  
    if(P[x] == x) {  
        return x;  
    }  
    return Find(P[x]);  
}  
  
int Find(int x) {  
    if(P[x] == x) {  
        return x;  
    }  
    return P[x] = Find(P[x]);  
}  
  
void Union(int x, int y) {  
    int rootx = Find(x);  
    int rooty = Find(y);  
    if(rootx != rooty) {  
        P[rootx] = rooty;  
    }  
}
```

第 9 页

2.4 MST（最小生成树）

2.4.1 Prim算法

用法

在稠密图中求解最小生成树，从任意结点出发，通过维护当前连通块到未连通结点的最小边权，逐步扩展连通块。时间复杂度 $\mathcal{O}(m \log n)$ （优先队列实现）。

实现步骤

1. 初始化距离数组 `dist[]` 为无穷大，选择任意起点（如结点1），将其距离设为0
2. 使用优先队列（小顶堆）存储（距离，结点），将起点入队
3. 用 `vis[]` 数组维护已加入最小生成树的结点

4. 每次取出队首距离最小的未访问结点，标记为已访问，累加其距离到总权值
5. 遍历该结点的所有邻接点，若邻接点未访问且当前边权小于其 `dist`，更新 `dist` 并入队

完整模板

```

#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
const int INF = 0x3f3f3f3f;

vector<vector<pair<int, int>>> G;
vector<int> dist;
vector<bool> vis;

i64 prim(int n, int start = 1) {
    dist.assign(n + 1, INF);
    vis.assign(n + 1, false);
    dist[start] = 0;
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
    pq.emplace(0, start);
    i64 total = 0;
    int cnt = 0;
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (vis[u]) continue;
        vis[u] = true;
        total += d;
        cnt++;
        for (auto [v, w] : G[u]) {
            if (!vis[v] && w < dist[v]) {
                dist[v] = w;
                pq.emplace(dist[v], v);
            }
        }
    }
    return cnt == n ? total : -1;
}

int main() {
    int n, m;
    cin >> n >> m;
    G.resize(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        G[u].emplace_back(v, w);
        G[v].emplace_back(u, w);
    }
    i64 ans = prim(n);
    if (ans == -1) cout << "orz" << endl;
}

```

```
else cout << ans << endl;  
return 0;  
}
```

2.4.2 Kruskal算法

用法

在**稀疏图**中更优，通过将所有边按权值排序，结合并查集逐步合并连通分量，每次选择连接两个不同连通分量的最小边。时间复杂度 $\mathcal{O}(m \log m)$ （主要为边排序的时间）。

实现步骤

1. 定义边的结构体，包含起点、终点、权值
2. 将所有边按权值从小到大排序
3. 初始化并查集，每个结点独立成一个集合
4. 遍历排序后的边，若边的两个端点不在同一连通分量，则合并它们，并累加该边权值到总权值
5. 当合并次数达到 $n-1$ 时（已形成生成树），提前退出

完整模板

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

struct Edge {
    int u, v, w;
    Edge(int u, int v, int w) : u(u), v(v), w(w) {}
    bool operator<(const Edge& other) const {
        return w < other.w;
    }
};

vector<Edge> edges;
vector<int> P;

int Find(int x) {
    if (P[x] != x) P[x] = Find(P[x]);
    return P[x];
}

i64 kruskal(int n) {
    sort(edges.begin(), edges.end());
    P.resize(n + 1);
    for (int i = 1; i <= n; i++) P[i] = i;
    i64 total = 0;
    int cnt = 0;
    for (auto& e : edges) {
        int rootu = Find(e.u);
        int rootv = Find(e.v);
        if (rootu != rootv) {
            P[rootu] = rootv;
            total += e.w;
            cnt++;
            if (cnt == n - 1) break;
        }
    }
    return cnt == n - 1 ? total : -1;
}

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        edges.emplace_back(u, v, w);
    }
}
```



```
}
i64 ans = kruskal(n);
if (ans == -1) cout << "orz" << endl;
else cout << ans << endl;
return 0;
}
```

2.5 LCA（最近公共祖先）

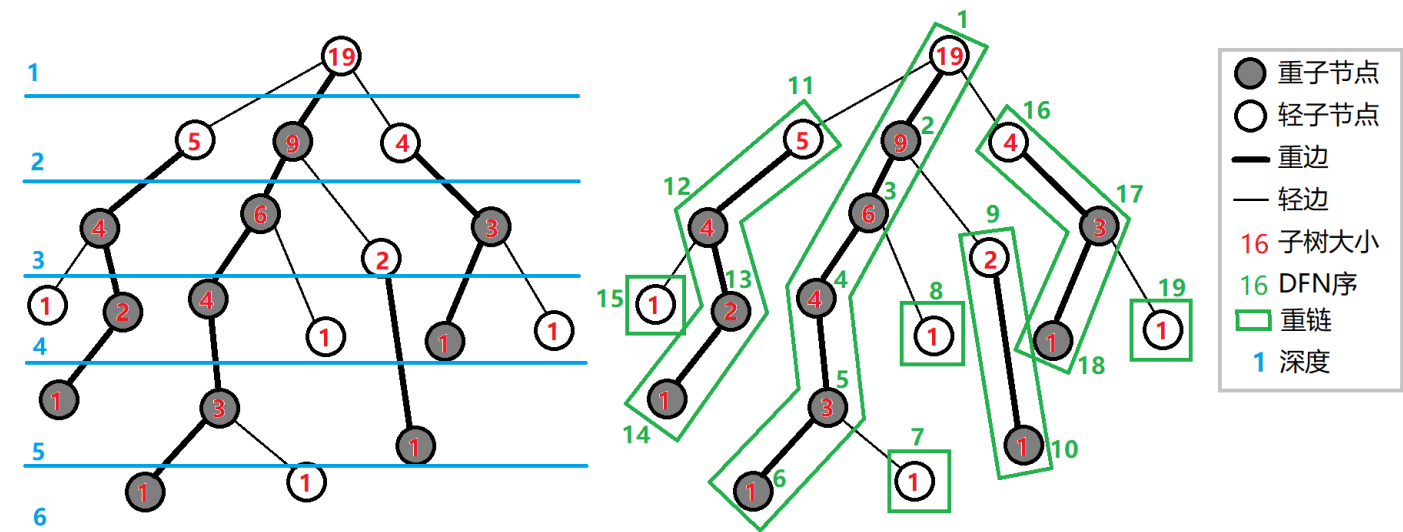
2.5.1 树链剖分

用法

预处理 $O(n)$ ，查询 $O(\log n)$ ，通过重链剖分将树划分为若干重链，快速跳转到LCA。

实现步骤

1. 第一个DFS（dfs1）：处理父结点father、深度depth、子树大小size、重子结点son
2. 第二个DFS（dfs2）：创建链顶top，重子结点的链顶为父结点的链顶，轻子结点的链顶为自己
3. 寻找LCA：
 - 若二者不在同一个重链上，让链顶深度大的点跳转到他链顶的父结点
 - 直到二者在同一个重链上，深度更小的点为LCA



完整模板

```

vector<vector<int>> G;
vector<int> fa, dep, siz, son, top;

void dfs1(int u, int f) {
    fa[u] = f, dep[u] = dep[f] + 1, siz[u] = 1;
    for (auto v : G[u]) {
        if (v == f) continue;
        dfs1(v, u);
        siz[u] += siz[v];
        if (siz[v] > siz[son[u]]) son[u] = v;
    }
}

void dfs2(int u, int f) {
    if (son[f] == u)
        top[u] = top[f];
    else
        top[u] = u;
    for (int v : G[u]) {
        if (v != f)
            dfs2(v, u);
    }
}

int LCA(int a, int b) {
    while(top[a] != top[b]) {
        if (dep[top[a]] > dep[top[b]]) a = fa[top[a]];
        else b = fa[top[b]];
    }
    return dep[a] <= dep[b] ? a : b;
}

```

2.5.2 倍增

用法

预处理 $O(n\log n)$ ，查询 $O(\log n)$ ，利用二进制思想，通过倍增表快速跳转到祖先结点。

实现步骤

1. DFS：处理父亲函数 $fa[0][i]$ ($2^0 = 1$) 和深度 dep

2. 倍增表初始化：第 2^k 祖先是第 $2^{(k-1)}$ 祖先的第 $2^{(k-1)}$ 祖先

3. LCA函数：

- 先将二者跳到同一深度
- 若相同则直接返回
- 否则从 2^k 到 2^0 尝试跳转，直到找到LCA

完整模板

```
vector<vector<int>> Edge;
vector<vector<int>> fa;
vector<int> dep;
int n;

void dfs(int u, int f) {
    fa[0][u] = f;
    dep[u] = dep[f] + 1;
    for(auto &i:Edge[u]) {
        if(i != f) dfs(i, u);
    }
}

void init() {
    for(int i = 1; i <= 20; i++) {
        for(int j = 1; j <= n; j++) {
            fa[i][j] = fa[i - 1][fa[i - 1][j]];
        }
    }
}

int LCA(int a, int b) {
    if(dep[a] < dep[b]) swap(a, b);
    for(int i = 20; i >= 0; i--) {
        if(dep[fa[i][a]] >= dep[b]) a = fa[i][a];
    }
    if(a == b) return a;
    for(int i = 20; i >= 0; i--) {
        if(fa[i][a] != fa[i][b]) {
            a = fa[i][a];
            b = fa[i][b];
        }
    }
    return fa[0][a];
}
```

三、单调队列 & 单调栈 核心笔记

3.1 单调队列

用法

单调队列可用以解决滑动窗口最值问题

实现步骤

单调队列的本质可以分为6步

1. 从左到右遍历整个队列
2. 每次遍历到新元素的时候分别考虑
3. 队头是否滑出范围 是则删除到不是为止
4. 队尾是否比新来的更小或者更大 是则删除到不是为止
5. 入队新元素
6. 此时队头就是窗口的最小值

完整模板

```
deque<int> dq;
for (int i = 0; i < n; i++) {
    if (!dq.empty() && dq.front() < i - k + 1) dq.pop_front();
    while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
    dq.push_back(i);
    if (i >= k - 1) cout << a[dq.front()] << " ";
}
```

3.2 单调栈

用法

找左右边界、下一个大小元素、矩形 / 接雨水、子数组贡献、嵌套括号

实现步骤

单调栈的本质可以分为7步

1. 从左到右遍历整个队列
2. 每次遍历到新元素的时候考虑
3. 新元素是不是比当前的栈头更大或者更小或其他
4. 如新元素是 则弹出头到不是为止
5. 入栈新元素
6. 此时栈头就是窗口的最小值
7. 答案数组记录下此时的位置即可

完整模板

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
int main() {
    i64 n;
    cin >> n;
    vector<i64> arr(n);
    vector<i64> ans(n, 0);
    for(auto &i:arr) {
        cin >> i;
    }
    stack<i64> S;
    for(i64 i = 0; i < n; i++) {
        while (!S.empty() && arr[S.top()] < arr[i]) {
            ans[S.top()] = i + 1;
            S.pop();
        }
        S.push(i);
    }
    for(auto &i:ans) {
        cout << i << ' ';
    }
}
```

四、ST表（倍增）

用法

用以维护静态可重复贡献RMQ，例如 MAX、MIN、GCD、LCM、|、&。

时间复杂度

- 预处理时间： $\mathcal{O}(n \log n)$
- 查询时间： $\mathcal{O}(1)$

核心思想

ST表运用了倍增的思想：

先维护每个位置从 2^0 到 2^n （小于最后位置）长度中的RMQ。

例子：数组 1 2 3 4 5 6 7 8，查询 2-6 只需 2-4 和 4-6 合并。

实现步骤

1. logn预处理

```
void pre() {
    logN[2] = 1;
    for(i64 i = 3; i < n; i++) {
        logN[i] = logN[i / 2] + 1;
    }
}
```

2. 初始化

```
void init() {
    for(i64 i = 0; i < n; i++) st[i][0] = arr[i];
    for(i64 i = 1; i <= logN[n]; i++) {
        for(i64 j = 0; j + (1 << i) <= n; j++) {
            st[j][i] = max(st[j][i - 1], st[j + (1 << (i - 1))][i - 1]);
        }
    }
}
```

3. 查询

```

i64 query(i64 l, i64 r) {
    i64 k = logN[l - r + 1];
    return max(st[l][k], st[r - (1 << k) + 1][k]);
}

```

完整板子

```

#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
i64 n;
vector<i64> logN;
vector<i64> arr;
vector<vector<i64>> st(100, vector<i64>(100));

void pre() {
    logN[2] = 1;
    for(i64 i = 3; i < n; i++) {
        logN[i] = logN[i / 2] + 1;
    }
}

void init() {
    for(i64 i = 0; i < n; i++) st[i][0] = arr[i];
    for(i64 i = 1; i <= logN[n]; i++) {
        for(i64 j = 0; j + (1 << i) <= n; j++) {
            st[j][i] = max(st[j][i - 1], st[j + (1 << (i - 1))][i - 1]);
        }
    }
}

i64 query(i64 l, i64 r) {
    i64 k = logN[l - r + 1];
    return max(st[l][k], st[r - (1 << k) + 1][k]);
}

int main() {
    cin >> n;
    arr.resize(n + 1);
    logN.resize(n + 1);
}

```

五、字符串算法（KMP + Manacher）

5.1 KMP算法

用法

用于单模式串匹配，可在主串中快速查找模式串的所有出现位置，避免暴力匹配的回溯，时间复杂度 $O(n + m)$ 。

实现步骤

对于一串字符串 如

匹配串: abcdabxa...

模式串: abcdabcy

可以发现 abcdab是匹配的

且abcdab的最长前后缀存在为ab

因为前面都匹配 只有从c-x不匹配

所以x前的肯定是ab

那就意味着可以从c-x开始比较 而不用从头开始遍历

完整模板


```

#include <bits/stdc++.h>
using namespace std;
int main() {
    string s1;
    string s2;
    cin >> s1 >> s2;
    int s2_len = s2.size();
    int s1_len = s1.size();
    vector<int> next(s2_len, 0);
    int i = 0, j = 1;
    while(i < s2_len && j < s2_len) {
        while (i > 0 && s2[i] != s2[j]) {
            i = next[i - 1];
        }
        if(s2[i] == s2[j]) i++;
        next[j] = i;
        j++;
    }
    i = 0;
    j = 0;
    while (j < s1_len) {
        while(i > 0 && s2[i] != s1[j]) {
            i = next[i - 1];
        }
        if(s2[i] == s1[j]) {
            i++;
        }
        if(i == s2_len) {
            cout << j - i + 2 << endl;
        }
        j++;
    }
    for(auto &i:next) {
        cout << i << ' ';
    }
}

```

5.2 Manacher (马拉车算法)

用法

用于在线性时间 $\mathcal{O}(n)$ 内求解字符串的最长回文子串长度。

实现步骤

1. 预处理字符串：插入 # 统一奇偶回文
2. 维护 `maxr`、`mid`、`ans`
3. 遍历中心，利用对称性扩展回文半径

完整模板

```
#include <bits/stdc++.h>
using namespace std;
string change(string &s) {
    string temp = "!#";
    for(auto &c:s) {
        temp += c;
        temp += '#';
    }
    temp += "^";
    return temp;
}

int manacher(string &s) {
    int maxr = 0, mid = 0, ans = 0;
    vector<int> p(s.size(), 0);
    for(int i = 0; i < s.size(); i++) {
        if(maxr >= i) {
            p[i] = min(p[2 * mid - i], maxr - i);
        }
        while(s[i - p[i] - 1] == s[i + p[i] + 1])
            p[i]++;
        if(i + p[i] > maxr) {
            mid = i;
            maxr = i + p[i];
        }
        ans = max(ans, p[i]);
    }
    return ans;
}

int main() {
    string s;
    cin >> s;
    s = change(s);
    cout << manacher(s);
}
```

六、数论基础

6.1 费马小定理笔记

1. 定义与基本形式

定义：若 p 是质数，且 $\gcd(a, p) = 1$ ，则：

$$a^{p-1} \equiv 1 \pmod{p}$$

一般形式：

$$a^p \equiv a \pmod{p}$$

2. 核心应用：模逆元

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

3. 快速幂模板

```
#include <iostream>
using namespace std;
using i64 = long long;

i64 fast_pow(i64 base, i64 exponent, i64 mod) {
    i64 result = 1;
    base %= mod;
    while (exponent > 0) {
        if (exponent % 2 == 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exponent /= 2;
    }
    return result;
}

i64 fermat_inverse(i64 a, i64 p) {
    return fast_pow(a, p - 2, p);
}

int main() {
    i64 a, p;
    cin >> a >> p;
    cout << fermat_inverse(a, p) << endl;
    return 0;
}
```

6.2 等比数列求和公式笔记

1. 基本公式

设等比数列首项为 a_1 , 公比为 q , 项数为 n 。

当 $q \neq 1$ 时:

$$S_n = \frac{a_1(1 - q^n)}{1 - q}$$

当 $q = 1$ 时:

$$S_n = na_1$$

2. 公式推导 (错位相减法)

$$\begin{aligned} S_n &= a_1 + a_1q + a_1q^2 + \cdots + a_1q^{n-1} \\ qS_n &= a_1q + a_1q^2 + \cdots + a_1q^{n-1} + a_1q^n \end{aligned}$$

两式相减:

$$S_n - qS_n = a_1 - a_1q^n$$

$$(1 - q)S_n = a_1(1 - q^n)$$

$$\therefore S_n = \frac{a_1(1 - q^n)}{1 - q}$$

3. 无穷等比数列 ($|q| < 1$)

$$S = \frac{a_1}{1 - q}$$

4. 常用变形

已知条件	公式
已知 a_1 、 a_n	$S_n = \frac{a_1 - a_nq}{1 - q}$

已知条件	公式
已知 $a_1、a_n$	$S_n = \frac{a_nq-a_1}{q-1}$

其中 $a_n = a_1q^{n-1}$ 。